Building ASP (Agent Semantic Protocol)

A Structure-Aware Document RAG Approach for the Enterprise

Kushal Sharma

2026-06-29

Table of Contents

Document control 2
Version history 2
Confidentiality 4
1. Executive summary 5
2. Why document structure matters 6
3. The 2026 landscape — what enterprises run today 6
3.1 Flat chunking + vector retrieval — the de-facto baseline 7
3.2 Hybrid retrieval — BM25 + dense vector + reranker 7
3.3 Synthetic-hierarchy and clustering-based methods — “hierarchical-looking” but built post-hoc 7
3.4 Graph-based RAG — entity graphs, not document graphs 7
3.5 Native-structure retrieval — the rare genuine cases 8
3.6 Reading the landscape 8
4. Positioning of ASP 8
How ASP retrieves 9
Path-augmented BM25 — the arithmetic 10
Contrast with §3 12
5. A representative experiment — *Adaptive Context Floor* (EE4-X1) 12
5.1 The problem this experiment addresses 12
5.2 Method 13
5.3 Gating — and its current limit 13
5.4 Principle 13
5.5 The expand procedure, in detail 13
5.6 Worked example — one query, end to end 16
5.7 Why this is a useful demonstration 19
6. Results 19

6.1 Baseline run (no context floor) — `rr_final` 20
6.2 Adaptive Context Floor active — `rr_ctx700` 20
6.3 Reading the tables 21
6.4 Structure-aware chunking baseline (EE-Struct) 21
6.5 Model-generalisation check — `llama_base` 24
7. Open work 25
7.1 EE4-X2 — Top-1 BM25 margin gate (next experiment) 26
8. Appendices 27
8.1 Glossary of identifiers 27
8.2 Survey matrix 29

Document control

Field	Value
Title	Building ASP (Agent Semantic Protocol): A Structure-Aware Document RAG Approach for the Enterprise
Author	Kushal Sharma
Status	Internal — for reviewer circulation
Classification	Confidential — pre-publication research
Current version	v2.1
Current date	2026-07-10
Branch of record	`experiment/doc-tree-retrieval-redesign`
Project	ASP — Agent Semantic Protocol (research programme)

Version history

Version	Date	Author	Summary of changes
v1	2026-06-26	Kushal Sharma	Initial case study circulated for reviewer feedback. Covered the §3 landscape, §4 ASP positioning, the EE4-X1 Adaptive Context

Version	Date	Author	Summary of changes
			Floor experiment, and the `rr_final` / `rr_ctx700` results.
v2	**2026-06-29**	**Kushal Sharma**	**Adds §4 *Path-augmented BM25 — the arithmetic* (formula, parameters, knob analysis); §5.5 *The expand procedure, in detail* (pseudocode, deep-tree behaviour, token-budget conflicts); §5.6 *Worked example — one query, end to end* (real BM25 scores, expansion trace, final bundle); §6.4 *Structure-aware chunking baseline (EE-Struct)* (three-way comparison — ASP, flat hybrid, and a structure-aware-chunking flat baseline — on CUAD and MAUD); §7.1 *EE4-X2 — Top-1 BM25 margin gate (next experiment)*; §8.1 *Glossary of identifiers* (defines the `EE/X` experiment-code scheme and the `rr_` run-tag scheme for the identifiers referenced in this document).**

Version	Date	Author	Summary of changes
			Updates the executive summary to surface the MAUD-conditional caveat and the EE-Struct finding. Adds an upfront note in §6 reconciling the three run tags (`rr_final`, `rr_ctx700`, `rr_struct_baseline`).
v2.1	2026-07-10	Kushal Sharma	Adds §6.5 *Model-generalisation check — `llama_base`*: the §6.1 harness re-run with the generation model swapped to Llama-3.3-70B (no retrieval change), confirming every ASP-vs-flat directional finding reproduces across a third base model, with MAUD reaching near-parity without the context floor. Adds the `llama_base` glossary entry. Links the full scorecard in `llama_base_report.md`.

Confidentiality

This document describes original, unpublished research on the **Agent Semantic Protocol (ASP)**, including a structure-aware document retrieval method (path-augmented BM25 + tree traversal + adaptive context floor + authority-ordered bundle assembly) that may form the basis of one or more patent applications.

Do not redistribute outside the named reviewer audience without the author's written consent. Do not post excerpts, formulas, pseudocode, or benchmark tables to public forums, mailing lists, or third-party LLM endpoints. Forwarding within the named organisation for internal review is permitted.

1. Executive summary

- **The state of document RAG in 2026 is monoculture.** Every major cloud (Azure, AWS, Google, Snowflake, Databricks), every major vector database (Pinecone, Weaviate, Elastic), and the dominant legal and enterprise AI products (Glean, CoCounsel, Harvey, NetDocuments) ship variants of the same recipe: cut documents into fixed-size chunks, index them with a hybrid of BM25 and dense vectors, and rerank.
- **The "hierarchical RAG" label is doing a lot of work it does not earn.** Most things sold as hierarchical are either size-based parent/child token windows (AWS Bedrock), summary trees built post-hoc by clustering (RAPTOR, GraphRAG community detection), or layout-aware chunking that still feeds a flat vector index (Azure Document Layout, Docling, Unstructured). None of these is the document's own structure.
- **A small set of methods do retrieve directly over a document's native heading and section tree** — notably PageIndex, HiQA, RDR2, LAD-RAG, and Google's layout-aware mode inside Vertex AI Search — but none is a mainstream production default in 2026, and the most production-ready option among them sits behind a premium Google Cloud SKU at roughly **$4 per 1,000 enterprise search queries**, plus an additional LLM grounding charge for RAG-style answers.[1]
- **ASP — the *Agent Semantic Protocol* — is being built to occupy that gap.** It retrieves and assembles in the document's native coordinate system — parsed tree, heading-path BM25, parent/sibling expansion, authority-ordered context bundles with structural headers — instead of over fixed-size chunks, and it runs on self-hosted infrastructure rather than a metered SaaS endpoint.
- **One representative experiment in this report — EE4-X1, the *Adaptive Context Floor*** — illustrates what structure-aware retrieval enables in practice: it expands an undersized bundle by walking the document's own parent and sibling edges until a minimum-context floor is met. On the MAUD legal multiple-choice benchmark, **with the floor active**, it converted a measurable deficit into a measurable lead against the flat-hybrid baseline (§6.2). **Without the floor — the EE-Struct configuration in §6.4 — ASP trails flat on MAUD by 2–4 pp.** So the MAUD lead is contingent on EE4-

[1] Vertex AI Search publishes per-query pricing across Standard and Enterprise tiers, with an additional LLM Add-On for grounded generation. Confirm the exact figure for the current quarter on the Vertex AI Search pricing page before quoting externally; pricing has been restructured across multiple rebrands (Discovery Engine → Vertex AI Search → Gemini Enterprise) and is best read from the live page on the day of the talk.

X1 firing, and EE4-X1 itself fires on bundle size rather than retrieval ambiguity (admittedly the wrong signal — §7.1 replaces it with a top-1 BM25 margin gate). On the CUAD contract-extraction benchmark the floor fires more eagerly than it should and mildly dilutes spans — the same gating limitation, addressed by the same follow-up.

- **A structure-aware chunking baseline does not capture the gain.** A third backend (EE-Struct, §6.4) feeds ASP's own heading-path-prefixed section chunks into the same hybrid retriever as flat — the comparison reviewers correctly ask for. On CUAD it sits neutral with plain flat (within 1–2 pp); on MAUD it actively *underperforms* flat by 8–10 pp because clause-level chunks fragment too aggressively. ASP's win on CUAD (+15 pp EM vs both baselines in the harness-aligned `rr_struct_baseline` run; the earlier `rr_final` and `rr_ctx700` runs in §6.1/§6.2 show +14–18 pp on the same direction) is therefore not explained by heading-path enrichment alone — tree traversal, parent/sibling expansion, and authority-ordered bundle assembly are doing the work.

---

2. Why document structure matters

Document RAG is the problem of answering a question by retrieving evidence from one or more long, structured documents — contracts, policies, regulatory filings, multi-section reports — and presenting that evidence to a language model.

The lowest-common-denominator solution is to chop each document into fixed-size token windows, embed each window into a vector, and at query time return the windows whose embeddings sit closest to the query. It is cheap, generic, and works tolerably on short corpora. It is also the wrong shape for the documents enterprises actually run on. A 200-page credit agreement is not a bag of 500-token chunks. It is a tree — *Article* → *Section* → *Sub-clause* → *Paragraph* — and the answer to almost every business question depends on where in that tree the evidence sits, what sits next to it, and which source it came from.

The positioning of this work is simple. **Retrieval that operates on the document's parsed structure should outperform retrieval that ignores it, and it should do so on infrastructure the enterprise owns.** Section 3 surveys what the market is shipping today against that bar. Section 4 sets out where ASP differs. Sections 5–6 show the supporting evidence.

---

3. The 2026 landscape — what enterprises run today

The methods deployed and discussed in 2026 fall into five categories. Each is summarised below with named methods, primary citation, and an explicit note on whether the method uses the document's native heading/section structure as the retrieval signal — as opposed to using structure only as a parsing or chunking heuristic.

3.1 Flat chunking + vector retrieval — the de-facto baseline

The default RAG pipeline in 2026 is fixed-size token windowing followed by dense vector retrieval. Pinecone advises that *"fixed-sized chunking will be the best path in most cases."* Weaviate recommends a 512-token / 50-100-overlap baseline. Snowflake Cortex Search caps chunks at 512 tokens. Azure AI Search's default Text Split skill uses 2,000-char pages with 500-char overlap. ColBERT and ColPali sit one layer below as passage-level and page-image alternatives but do not change the underlying chunking premise. **Native structure used: no.**

3.2 Hybrid retrieval — BM25 + dense vector + reranker

The enterprise default upgrades 3.1 by running BM25 in parallel with a dense vector retriever, fusing the two ranked lists (typically with Reciprocal Rank Fusion), and reranking the top candidates with a cross-encoder. This pattern was popularised by BEIR and is now the implementation behind Databricks Mosaic AI Vector Search, Elastic semantic_text + ELSER, Snowflake Cortex Search, Glean enterprise search, Anthropic Contextual Retrieval, and the LangChain and LlamaIndex reference stacks. **Native structure used: no — chunks are still flat, and structure does not enter the ranker.**

3.3 Synthetic-hierarchy and clustering-based methods — "hierarchical-looking" but built post-hoc

These methods construct a tree, but the tree they build is not the document's tree.

- **RAPTOR** (Sarthi et al., ICLR 2024) recursively embeds, clusters, and summarises chunks bottom-up.
- **LlamaIndex DocumentSummaryIndex** creates a two-level *document-summary → chunks* construct.
- **LangChain ParentDocumentRetriever** splits parents and children using a recursive character text splitter.
- **AWS Bedrock Hierarchical Chunking** is a 1,500-token-parent / 300-token-child token-window scheme.

Native structure used: no — the hierarchy is synthesised from text, not lifted from the document.

3.4 Graph-based RAG — entity graphs, not document graphs

- **Microsoft GraphRAG** slices into TextUnits, LLM-extracts entities and relations, and Leiden-clusters the resulting entity graph into community summaries.
- **LightRAG** and **HippoRAG** run LLM OpenIE over chunks.
- **Neo4j GraphRAG SimpleKGPipeline** performs schema-guided LLM extraction.
- **KAG** (Ant Group) is the closest hybrid — it adds layout analysis to mutual chunk-to-graph indexing — but the graph remains schema- and LLM-driven.

Native structure used: no — the graph is over entities the LLM extracted, not over the document's sections.

3.5 Native-structure retrieval — the rare genuine cases

A small set of methods do retrieve over the document's parsed tree as the primary signal:

- **PageIndex** (VectifyAI, 2024–2025) builds a TOC-like tree and navigates it by LLM reasoning. Vector-free. Per-query latency of 30–120 seconds keeps it in research and proof-of-concept use.
- **HiQA** (Chen et al., 2024) parses sections and cascades the heading metadata into each chunk before retrieval, so the vector retriever sees the hierarchy.
- **RDR2** (Xu et al., EMNLP 2025 Findings) trains a router that navigates parsed document structure trees as a primary retrieval task.
- **LAD-RAG** constructs a layout graph of visually-rich documents and retrieves against it.
- **Google Vertex AI Search — layout-aware chunking mode** is the closest production case: chunks are bounded by layout entities and can prepend ancestor headings, and the service is offered behind a premium per-query SKU with an additional LLM grounding charge.

Layout-aware parsers — LlamaParse, Azure AI Document Intelligence Layout, Docling HierarchicalChunker, Unstructured's `chunk_by_title`, Marker, and Adobe PDF Extract — preserve heading structure in metadata but emit a flat chunk stream that the downstream retriever still treats as a bag. **Structure-aware chunking is not the same as structure-aware retrieval.**

3.6 Reading the landscape

The picture is consistent across all five categories. Mainstream production stacks and the open-source frameworks that feed them retrieve over flat, chunked representations. The structure of the document is, at best, used to draw the boundaries of the chunks; it is not the signal the retriever ranks against. The methods that do treat the document's parsed tree as a first-class retrieval surface are concentrated in late-2024 and 2025 research releases and in one premium SaaS SKU. **There is open ground between research and production for a structure-aware document RAG approach that an enterprise can own and operate on its own infrastructure.**

---

4. Positioning of ASP

ASP is being built to occupy that open ground. Its design commitments are deliberately different from the 2026 mainstream:

1. **Native-tree retrieval, not chunking.** The parsed document tree — articles, sections, sub-clauses, paragraphs — is the unit of retrieval, ranking, and citation. There is no fixed-size token window in the index.
2. **Structural bundle assembly, not concatenate-and-truncate.** Context is assembled from tree nodes in an order that respects source authority, with explicit token budgeting, structural headers, and an audit trail of anything that was dropped.
3. **Self-hosted, not metered.** ASP runs on infrastructure the enterprise controls. It does not depend on a per-query SaaS endpoint whose unit economics scale linearly with usage. The closest production-ready structure-aware alternative — Vertex AI Search with the LLM Add-On — is offered at roughly $4 per 1,000 enterprise-tier queries plus the grounding charge,[2] which compounds quickly for a workflow that issues thousands of retrieval calls per day.
4. **Same-condition benchmarking against the prevailing pattern.** Every claim in this work is measured against a fair flat-hybrid baseline (BM25 + dense vector + cross-encoder reranker) running in the same process, on the same questions, on the same documents.

How ASP retrieves

- **Parsing.** A dispatcher detects numbering and heading conventions and emits a hierarchical *DocumentTree* — articles, sections, sub-clauses, paragraphs. Where the dispatcher cannot resolve a structure, the pipeline falls back to paragraph-split section nodes so the downstream contract is preserved. Multi-document corpora can supply pre-built node dictionaries.
- **Node.** A *TreeNode* addresses one structural unit. It carries a stable `node_id`, `node_type`, `doc_slug`, title, full `heading_path` (e.g. *"Article 7 > Conditions to Closing"*), depth, content, optional `clause_id`, parse confidence, and token/char counts, with parent and child links so the tree is navigable up and sideways. **The node — not a fixed-size chunk — is the unit of retrieval, ranking, and citation.**
- **Retrieval traversal.** (i) Path-augmented BM25 scores every node, weighting matches in the heading path as well as the body, with a depth multiplier tuned to whether the corpus favours leaf clauses or section paragraphs. (ii) A candidate-seeding step selects high-scoring nodes while ensuring multi-source coverage so a single dominant document cannot monopolise the set. (iii) A tree-traversal expansion walks ancestors and siblings of the seeds to add structurally adjacent context that pure BM25 would miss. (iv) A cross-encoder reranks the expanded set and retains the top-K.

---

[2] Vertex AI Search publishes per-query pricing across Standard and Enterprise tiers, with an additional LLM Add-On for grounded generation. Confirm the exact figure for the current quarter on the Vertex AI Search pricing page before quoting externally; pricing has been restructured across multiple rebrands (Discovery Engine → Vertex AI Search → Gemini Enterprise) and is best read from the live page on the day of the talk.

- **Bundle assembly.** The surviving nodes are assembled into an ASP *ContextBundle*. Nodes are sorted first by source `authority_level` and only secondarily by retrieval rank, so authoritative sources lead the prompt. For each node, a paragraph extractor pulls the query-relevant paragraphs, and a compact header is emitted naming the source, the leaf section, and the clause ID — `[<SourceName> <LeafSection> | <ClauseID>]`. Tokens are budgeted explicitly against a hard cap; if anything is dropped, the bundle records `compression_applied` and surfaces what was dropped. The protocol forbids silent truncation.

Path-augmented BM25 — the arithmetic

Path-augmented BM25 scores *TreeNode* objects rather than fixed-width chunks: each node carries its content plus its heading path ("Article 5 > Termination > Notice"), and the retriever ranks whole nodes against the query. The score is built in four layers — a BM25 term over the concatenation of content tokens and path tokens, an empty-container child aggregation step that lets pure section headers borrow their immediate children's tokens so they can still score, an additive heading-path overlap boost that fires only when the query and the path share ≥2 tokens, and a multiplicative depth bias that nudges the retriever toward leaves on contract corpora and toward shallow nodes on multi-hop QA. The four layers are composed in the order shown below; the parameters (k_1=1.5, b=0.75, heading-boost weight 0.5/token, depth multipliers ×1.2 / ×0.9 and ×1.1 / ×1.0) are the ASP-specific design choices.

```
Tokenisation (same for query and node content):
  drop tokens of length ≤ 2 and standard English stopwords
  (a, an, the, is, are, of, to, at, …)

Per-node document representation:
  content_tokens = normalise(title + " " + content)
  path_tokens    = normalise(heading_path)

Empty-container child aggregation (lets a section header whose
only payload is e.g. a table still score):
  if content_tokens is empty and node has child_node_ids
                              and a nodes_dict lookup is available:
    for each child with token_count > 0:
        extend content_tokens with normalise(child.title + " " + child.conten
t)

Per-node document representation (final):
  doc_tokens = content_tokens + path_tokens
  (the heading-path tokens are appended once — this is the
   "half-field-weight" effect: they contribute to TF without
   inflating doc length more than necessary)

IDF over the corpus of N nodes:
  df(t) = Σ_nodes [t ∈ content_set] + 0.5 · Σ_nodes [t ∈ path_set \ content_s
```

```
et]
  IDF(t) = log( (N + 1) / (df(t) + 1) )
  (path tokens count 0.5 to df — they help discrimination
   without inflating IDF for generic headings like "Article")

Standard Okapi BM25 with k₁ = 1.5, b = 0.75:
  TF(t, d) = f(t,d) · (k₁ + 1) / ( f(t,d) + k₁ · (1 − b + b · |d| / avg_dl) )
  BM25(q, d) = Σ_{t ∈ q} IDF(t) · TF(t, d)

Heading-path overlap boost (fires only when ≥2 query tokens overlap):
  overlap = |query_token_set ∩ path_token_set|
  heading_boost = 0.5 · overlap     if overlap ≥ 2
                = 0                  otherwise

Depth multiplier (prefer_leaf is set per-corpus):
  prefer_leaf = True   (contract corpora — CUAD, MAUD)
    depth_mult = 1.2   if depth ≥ 2
                 0.9   if depth < 2     ← penalises shallow section headers

  prefer_leaf = False  (HotpotQA, EW1, EW2)
    depth_mult = 1.1   if depth == 1
                 1.0   otherwise

Final node score:
  score(node, query) = ( BM25(q, doc_tokens) + heading_boost ) · depth_mult
```

What the knobs do

The depth multiplier is the weakest of the three ranking signals (the child-aggregation step is a precondition for scoring rather than a ranking signal — it decides whether an empty-container node enters the comparison at all). Its full dynamic range on contract corpora is 1.2/0.9 ≈ 1.33×, so it can only flip rankings between two nodes whose underlying BM25 + heading-boost scores are already within ~25% of each other. We did not run a dedicated `prefer_leaf` ablation on the five corpora in this study, so we cannot give a head-to-head Δ-EM number; based on the 1.33× dynamic range relative to the typical BM25 score gaps we observed in the per-query traces on CUAD/MAUD/HotpotQA/EW1/EW2, our working assumption is that it acts as a tiebreaker on a minority of queries and that turning it off would shift accuracy within noise — but this remains an empirical gap to close, not a demonstrated result. On these corpora the dominant signal is plain BM25 over `doc_tokens`; `depth_mult` is configured to bias toward leaves (CUAD/MAUD) or toward H1 anchors (HotpotQA, where the article title is usually the right hop). The heading-path overlap boost is the more decisive structural signal: +0.5 per overlapping token, gated at a minimum of 2 overlaps, which corresponds to ~10–25% of a typical BM25 raw score on these corpora. It bites hardest on structured documents where the question text repeats the section heading verbatim ("What is the *governing law*?" against a node on the "Governing Law" path) — exactly the CUAD/MAUD/EW shape. The half-weight on path tokens in the IDF denominator is what keeps generic headings like "Article" or "Section" from dominating;

without it, every contract chunk would look equally relevant to any query that mentioned an article number. The empty-container child aggregation, finally, is what keeps section-header nodes whose only payload is a child table or a child list from being silently dropped from the ranking: without it, those headers would tokenise to nothing and score 0, so any query whose answer lives under such a header would have to hit the child node directly.

Where this sits in §3's taxonomy

This formula is **structure-aware retrieval**, not structure-aware chunking: the heading path enters the ranking function as both a field (via `doc_tokens`) and a separate boost term, and ranking is performed over whole tree nodes. The contrast is with approaches like HiQA, which prefix the heading path onto flat fixed-size chunks and then run flat retrieval over the resulting strings — there the heading affects retrieval only through the chunk's bag-of-words, never as an explicit structural signal, and the unit of ranking remains a chunk rather than a node. That distinction is exactly what the EE-Struct experiment in §6.4 isolates by pitting this path-augmented BM25 against a structure-aware-chunking baseline.

Contrast with §3

The flat baseline indexes fixed-size token windows and assembles context by string concatenation plus truncation, with no awareness of section, clause, or source authority. ASP retrieves and assembles in the document's native coordinate system. Synthetic-hierarchy methods (RAPTOR, GraphRAG, LlamaIndex DocumentSummaryIndex) build a tree but not the document's tree. Layout-aware parsers (LlamaParse, Docling, Vertex's layout chunker) preserve structure as metadata but the downstream retriever still runs over a flat list. ASP is closest in spirit to PageIndex and RDR2 — primary retrieval over a parsed native tree — but uses BM25 + tree-expansion + cross-encoder rather than LLM tree-walk, which sidesteps PageIndex's latency problem.

ASP carries additional, orthogonal experiments at the prompt and seeding layers; those are out of scope for this report.

---

5. A representative experiment — *Adaptive Context Floor* (EE4-X1)

One experiment illustrates what becomes possible once retrieval operates on the document's structure rather than on a flat chunk stream.

5.1 The problem this experiment addresses

Some single-source classification and clause-comparison tasks fail not because retrieval picks the wrong clause, but because retrieval picks the *right* clause in isolation. The model sees one ~190-token clause and cannot distinguish near-neighbour candidates (for example, *"Mixed Cash/Stock"* versus *"Mixed Cash/Stock: Election"*) because the discriminating context lives in the *sibling* clauses it never saw.

We refer to this failure mode as **context starvation**. The hypothesis is that lifting the bundle above a minimum-context floor — by adding the parent section and its other children — closes the gap without any help from BM25 scores and without any dataset-specific signal.

5.2 Method

A `min_context_tokens` floor was added to the bundle assembler. After the normal authority-sorted, budget-bounded assembly pass, if the assembled bundle is below the floor and the full node graph is available, the assembler runs a structure-only BFS expansion: it enqueues the parent of each included node, then the parent's other children, and adds nodes until the floor is met or the hard token budget would be exceeded. Expansion is guided exclusively by tree topology — no BM25 scores, no query re-analysis, no dataset flags. The floor is plumbed through the pipeline as `min_context_tokens` and exposed on the runner as `--min-context-tokens`. The benchmark sweep ran at `min_context_tokens=700`. The full procedure is given in pseudocode in §5.5.

5.3 Gating — and its current limit

The expansion has no explicit dataset gate. The only built-in gate is implicit: the floor fires only when the post-budget bundle is below `min_context_tokens`. On multi-source corpora — where the assembled bundle is already well above 700 tokens — the floor never fires, so the mechanism is a no-op. The mechanism therefore activates exactly where it was designed to: on single-source corpora whose retrieval converges to a small number of short clauses.

The empirical result confirmed this gate is too permissive in one case: CUAD is single-source with short bundles (~240 tokens), so the floor fires — but CUAD is a span-extraction task where adding parent and sibling clauses dilutes the precise extraction span. The next experiment in the sequence (EE4-X2) replaces the implicit gate with a **top-1 BM25 margin** test (how cleanly the best candidate beats the runner-up), so the floor only triggers when retrieval is genuinely non-discriminating. See §7.1 for the mechanism and the rationale for choosing margin over a full score-variance signal.

5.4 Principle

> When the assembled bundle is too small to distinguish a candidate from its near-neighbours, expand along the document's own structural edges — parent first, then siblings — until the model has enough surrounding clauses to discriminate, never exceeding the hard token budget.

5.5 The expand procedure, in detail

Triggering condition and BFS order

The expand procedure fires only when the assembled bundle is below a configured `min_context_tokens` floor after the initial top-k nodes have been extracted and added (the default floor is 0, which disables expansion). The triggering check is a single comparison:

`running_tokens < min_context_tokens`, evaluated after the top-k pass. When the floor is unmet, the procedure seeds a BFS queue with the immediate parents of every already-included node, then walks the tree breadth-first — dequeueing a candidate, attempting to add it under the token budget, and then enqueuing that candidate's children. The BFS climbs exactly one level upward (to the parents of the original includes) and from there walks strictly *downward* through siblings and their descendants; it never re-ascends to a grandparent. The full procedure is given in pseudocode below — the algorithm is the protocol contract; the implementation is a faithful Python rendering of that contract.

```
function expand_to_floor(included, running_tokens, nodes_dict,
                         q_tokens, token_budget, min_context_tokens):

    if running_tokens >= min_context_tokens or nodes_dict is empty:
        return (included, running_tokens)        # no-op, floor already met

    seen_ids := { n.node_id for (n, _) in included }
    queue    := []

    # Seed: immediate parents of every already-included node
    for (node, _) in included:
        pid := node.parent_node_id
        if pid exists and pid not in seen_ids and pid in nodes_dict:
            queue.append(nodes_dict[pid])

    visited_parents := set()

    while queue is not empty and running_tokens < min_context_tokens:
        candidate := queue.pop_front()
        if candidate is None or candidate.node_id in seen_ids:
            continue

        # Reduce candidate to query-relevant paragraphs (may return full node
 text on weak overlap)
        extracted := extract_relevant_paragraphs(candidate, q_tokens)
        cost      := count_tokens(extracted) + HEADER_TOKEN_OVERHEAD  # +100

        # Hard budget check — never split, never truncate
        if running_tokens + cost <= token_budget:
            included.append((candidate, extracted))
            running_tokens += cost
            seen_ids.add(candidate.node_id)
        # else: skip this node but continue the BFS

        # Enqueue this candidate's own children (siblings of the original inc
lude)
        if candidate.node_id not in visited_parents:
            visited_parents.add(candidate.node_id)
            for cid in candidate.child_node_ids:
```

```
                if cid not in seen_ids and cid in nodes_dict:
                    queue.append(nodes_dict[cid])

    return (included, running_tokens)
```

Deep-tree behaviour

Consider a 200-page agreement parsed to six nesting levels (`Agreement > Article > Section > Subsection > Clause > Sub-clause`), where the top-k retrieval surfaced one Sub-clause node at depth 6. The seed step enqueues the immediate parent — the Clause at depth 5 — and nothing else. The grandparent (Subsection at depth 4) is **not** touched at this point, and under the current implementation it never will be.

When the BFS dequeues the Clause, it does two things in order: (a) it attempts to add the Clause itself under the budget, and (b) it enqueues the Clause's *children* (the Sub-clause siblings of the original include). It does **not** enqueue the Clause's own parent. As the BFS proceeds it only ever enqueues a candidate's `child_node_ids`; `parent_node_id` is consulted exactly once, during the seed step, and only on the *original* `included` list — not on nodes appended during expansion. The consequence is that the expand BFS climbs exactly one level upward (to the parents of the original includes) and from there walks strictly downward through siblings and their subtrees. Grandparents and higher ancestors are unreachable, irrespective of how short the bundle stays or how long the queue runs.

This is a known limitation. Reaching multi-level ancestors would require either (a) re-seeding the parents of expansion-added nodes back into the queue, or (b) enqueuing `candidate.parent_node_id` (gated by a `visited_parents` check) alongside `candidate.child_node_ids` during expansion. Neither is done today.

Across the CUAD, MAUD, EW1 and EW2 corpora tested, the floor (~700 tokens) is satisfied within one BFS level — the parent plus a handful of siblings — so the absence of a multi-level climb has not been a practical limit on real runs.

Token-budget conflicts

Suppose a single parent section weighs 5,000 tokens and `token_budget = 5,000` is already partially consumed (say `running_tokens = 800` from the original top-k include). The expand step does **not** see the 5,000-token raw section: an upstream paragraph extractor is run on the candidate first and typically returns only the query-relevant paragraphs — empirically a few hundred tokens, occasionally a thousand, and occasionally the full node text when query/node token overlap is weak or the node is small. The BFS cost is then `count_tokens(extracted) + HEADER_OVERHEAD`, where the header-overhead constant is fixed at 100 tokens per node (covering the compact header, clause id, effective date, and authority hint).

The budget check is strictly `running_tokens + cost <= token_budget`: pass and the node is appended; fail and the node is **skipped silently**, but the BFS continues to the next queued candidate rather than aborting. The assembler never splits a node — inclusion is

whole-or-nothing per node, with the upstream paragraph extractor providing the only form of sub-node selection. If every remaining candidate fails the budget check, the bundle finishes below the floor with no truncation and no partial inclusion.

Note that the bundle's `compression_applied` flag is **not** set in this case. That flag is only raised during the initial top-k assembly pass when a top-k node is dropped by the budget; the floor-expansion step never touches it. An under-floor bundle therefore carries no distinguishing flag in the returned tuple — downstream code that needs to identify under-floor bundles must compare `running_tokens` to `min_context_tokens` itself. (Adding an explicit `floor_unmet` flag is a planned protocol-level addition; tracking it is left to a follow-up revision.)

The fact that EE4-X1 triggers on bundle **size** rather than on retrieval **ambiguity** — and therefore over-expands on CUAD where the top-1 is already correct but small — is the limitation addressed by EE4-X2 (top-1 BM25 margin gate) in §7.1.

5.6 Worked example — one query, end to end

Framing

This trace walks a single MAUD row end to end on real pipeline output. ASP picked the correct option; the flat hybrid baseline on the same row returned a larger bundle and refused to answer. Numbers below were captured against the `rr_ctx700` run — the configuration where `min_context_tokens=700` is active and Phase B floor expansion fires when Phase A leaves the bundle under-filled. The point of the trace is to make the expand step, the arithmetic, and the failure mode of the flat baseline all inspectable on one row.

Query and document

Question ID: `maud_contract_49_Accuracy of Target R&W Closing Condition_10-7_361`

Question: "Accuracy of Fundamental Target R&Ws — Types of R&W: Opinion of Financial Advisor"

Candidates: - `[0]` Capitalization-Other, Authority, Approval, Enforceability, Organization, No MAE, Brokers' Fee - `[1]` Capitalization-Other, Authority, Approval, Enforceability, Organization, No MAE, Takeover Statutes, Rights Agreement, **Opinion of Financial Advisor** ← GOLD

Document: 8,090 characters → 39 nodes after parsing into the document tree.

BM25 + depth-adjusted scoring

Path-augmented BM25 (the four-layer scorer from §4) was run with `prefer_leaf=True`. Query tokens after stopword stripping: {`accuracy, fundamental, target, types, opinion, financial, advisor`}. Three of those (`opinion`, `financial`, `advisor`) appear in the §3.19 heading path; the other four do not appear anywhere in the document body.

Parameters: k_1=1.5, b=0.75, heading_boost=0.5/token, heading_boost_min_overlap=2, depth_mult (prefer_leaf=True): depth≥2 ×1.2 / depth<2 ×0.9.

rank	depth	base BM25	+heading boost	×depth mult	final	heading path
1	1	15.235	+1.50	×0.90	**15.062**	§3.19 Opinion of Financial Advisor
2	1	15.235	+1.50	×0.90	**15.062**	§3.19 Opinion of Financial Advisor (dup)
3	—	0.000	—	—	0.000	(37 other nodes, no query-token overlap)
4	—	0.000	—	—	0.000	—
5	—	0.000	—	—	0.000	—

Honest disclosure: 37 of 39 nodes scored exactly 0.0. The discriminating signal in this row is a single high-IDF path-token match on "Opinion of Financial Advisor". The depth multiplier (×0.9 for depth=1 with `prefer_leaf=True`) marginally **penalised** the §3.19 hits — pulling them from 16.735 down to 15.062 — but because the next-best score is 0.000, no multiplicative factor of any kind could flip the ranking. This is precisely the empirical test the v1 reviewer feedback called out: on this row the depth multiplier did not change anything.

Seed and rerank

- **Round-robin seed selection** (`cap=20`, per-source floor =4): 2 seeds returned — both §3.19 nodes, the only nodes with non-zero score.
- **Tree-traversal expansion** (pre-rerank): 2 seeds → 2 candidates. Siblings and parent fell below the score threshold, so no additional candidates were added at this stage.
- **Cross-encoder rerank** (top-6 cap): only 2 candidates available; both kept.

Bundle assembly and floor expansion

Bundle assembly first runs the budget-bounded sequential pass; if the assembled bundle is below the floor, the BFS floor-expansion logic from §5.5 fires. Header overhead is fixed at 100 tokens per included node, so per-node cost is `extracted_paragraph_tokens + 100`.

Phase A — sequential add up to `token_budget=5000`:

step	node	extracted tok	+ header	running
1	§3.19 Opinion of Financial Advisor (A)	10	+100	110
2	§3.19 Opinion of Financial Advisor (B)	10	+100	220

Phase A exits with `running=220 < floor=700`. Phase B fires.

Phase B — floor-expansion BFS:

step	action	cost	running
1	add §3 Representations and Warranties of the Target Company (container; parent of §3.19) — extracted=0, +header=100	+100	220 → 320
2	*(side effect of step 1: children of §3 enqueued into BFS queue — not a discrete step)*	0	320
3	add §7.3 Conditions to the Obligations…	+166	320 → 486
4	add §7.3 Conditions… (sub-paragraph)	+119	486 → 605
5	add §3.2 Capitalization	+108	605 → 713 ← floor met, stop

To be unambiguous: "parent" at step 1 means the §3 container that owns §3.19, not the document root. The container itself contributes zero extracted text — only its 100-token header — which is why the cost is a clean +100. **A note on §7.3 appearing in this BFS:** in this contract the parser flattens Article 7 nodes as siblings of §3.x under the same container (the dispatcher cannot resolve a separate Article-7 subtree from this contract's numbering), which is why §7.3 enters the §3-child queue rather than living in its own subtree. This is a parse artefact specific to this document, not a property of the BFS itself.

Final bundle and decision

- **ASP bundle:** 6 nodes, 713 tokens. Model picked option `[1]` — **correct**.
- **Flat hybrid baseline** (BM25 + dense vector + RRF + cross-encoder reranker — the BEIR-style stack defined in §6) on the same row in the `rr_ctx700` run: 1,284-token bundle, model answered `"None"` (refusal). The structure-aware `flat_struct` backend (§6.4) was run separately under the `rr_struct_baseline` configuration and is reported aggregately in that section, not per-row here.

What crossed the floor — §7.3 Conditions to the Obligations and §3.2 Capitalization — turned out to be the load-bearing context. Without expansion the model saw only two 10-token §3.19 fragments and could not discriminate between option `[0]`'s enumeration (ending at "Brokers' Fee") and option `[1]`'s enumeration (ending at "Opinion of Financial

Advisor”). The siblings supplied the enumeration context that made the categorical distinction visible.

What this trace tells us

The depth multiplier did **not** change the ranking in this row. That is the right thing to admit: one example does not generalise, and the multiplier’s effect is conditional. With `prefer_leaf=True`, depth<2 gets ×0.9 and depth≥2 gets ×1.2, so a deeper node only overtakes a shallower one when its base score is ≥ 0.75× the shallower node’s base score (since `1.2 × base_deep > 0.9 × base_shallow ⇔ base_deep > 0.75 × base_shallow`). A targeted experiment over rows with contested rankings — pairs whose base scores sit inside that 75% window — is the right way to measure the multiplier’s effect, and is currently unmeasured.

Expansion is deterministic given the parse and the question: same document, same query, same bundle. The Phase B stop arithmetic also behaved cleanly at the boundary — step 5 added §3.2 and that addition brought running to 713 ≥ 700, so BFS exited; the next queued candidate (the sixth) was never evaluated. The honest summary is that on this row most of the work was done by one high-IDF path-token match plus a clean floor expansion; the depth multiplier was inert.

5.7 Why this is a useful demonstration

This is a direct demonstration that document-tree structure can be used at bundle-assembly time to repair a failure mode that a flat-chunk retriever cannot even see, let alone fix. A chunk retriever has no parent and no siblings; ASP does.

---

6. Results

The flat baseline in these benchmarks is the standard hybrid stack: BM25 + dense vector (Chroma, BAAI/bge-small-en-v1.5) fused with Reciprocal Rank Fusion (k=60), reranked with `cross-encoder/ms-marco-MiniLM-L-6-v2`, K_retrieve=10 per arm, K_rerank=5. This is the same retrieval pattern enterprises deploy today (the BEIR / LangChain / LlamaIndex hybrid default), so the ASP-versus-flat comparison is a fair stand-in for *ASP versus what the enterprise actually runs*.

A note on runs and run tags before reading the tables. Three independent runs feed §6: `rr_final` (the original baseline without the context floor, §6.1), `rr_ctx700` (the same harness with `min_context_tokens=700` active, §6.2 and §6.3), and `rr_struct_baseline` (the three-way ASP / flat / flat_struct comparison reported in §6.4). All three share the same item sets, the same LLM endpoints, and the same evaluation harness, but were executed at different points across the project’s measurement timeline; the inference environment and rate-limiting configuration shifted between runs (this caveat already applied to §6.2 vs §6.1 and is called out in §6.3 bullet 5). The consequence is that **absolute** ASP and flat numbers shift modestly across the three tables — for example, ASP CUAD EM reads ≈87 in

`rr_final`, ≈80 with the floor in `rr_ctx700`, and 73.6 in the harness-aligned `rr_struct_baseline`; flat CUAD EM moves correspondingly (≈70 → ≈65 → ≈58). **The within-run ASP-versus-baseline delta is the load-bearing comparison and is preserved across all three runs:** ASP leads flat by +14 pp to +18 pp EM on CUAD in every run shown, regardless of absolute level. The same caveat applies to MAUD — different runs report different absolute accuracies for both ASP and flat, but the within-run delta is the comparable signal.

`n=500` per dataset for every cell. Numbers rounded to 1 decimal.

6.1 Baseline run (no context floor) — `rr_final`

Dataset	Model slot	Metric	ASP	Flat	Δ
CUAD	m2 (Nem)	EM	87.7	69.9	+17.8
CUAD	m2 (Nem)	F1	91.6	74.4	+17.2
CUAD	m4 (Mis)	EM	87.1	69.3	+17.8
CUAD	m4 (Mis)	F1	91.5	74.9	+16.5
CUAD	m5 (Nem)	EM	86.8	70.0	+16.8
CUAD	m5 (Nem)	F1	90.9	75.2	+15.8
CUAD	m6 (Mis)	EM	88.6	69.9	+18.7
CUAD	m6 (Mis)	F1	92.5	74.5	+18.0
MAUD	m3 (Nem)	Acc	36.2	39.4	−3.2
MAUD	m4 (Mis)	Acc	37.8	41.6	−3.8
HotpotQA	m1 (Nem)	F1	64.3	69.7	−5.4
HotpotQA	m2 (Mis)	F1	66.2	71.7	−5.6
HotpotQA	m3 (Nem)	F1	65.4	69.4	−4.0
HotpotQA	m5 (Nem)	F1	65.9	70.9	−5.0
EW1	m5 (Nem)	F1	11.8	7.6	+4.2
EW1	m6 (Mis)	F1	9.6	8.9	+0.7
EW2	m5 (Nem)	F1	10.6	8.1	+2.5
EW2	m6 (Mis)	F1	10.9	9.1	+1.8

6.2 Adaptive Context Floor active — `rr_ctx700`

Dataset	Model slot	Metric	ASP	Flat	Δ
CUAD	m5 (Nem)	EM	81.0	65.4	+15.6
CUAD	m5 (Nem)	F1	85.4	71.0	+14.4
CUAD	m6 (Mis)	EM	78.8	64.8	+14.0
CUAD	m6 (Mis)	F1	83.1	69.4	+13.7

Dataset	Model slot	Metric	ASP	Flat	Δ
MAUD	m3 (Nem)	**Acc**	**36.2**	**32.0**	**+4.2**
MAUD	m4 (Mis)	**Acc**	**36.4**	**35.2**	**+1.2**
EW1	m5 (Nem)	F1	7.7	5.4	+2.3
EW1	m6 (Mis)	F1	7.8	5.4	+2.4
EW2	m5 (Nem)	F1	10.5	7.8	+2.6
EW2	m6 (Mis)	F1	11.1	8.6	+2.5

HotpotQA is not included in the `ctx700` row because the assembled bundle already sits well above the 700-token floor (~1,200 tokens). The floor is a no-op there and the run was not repeated.

6.3 Reading the tables

- **MAUD flips from trailing to leading.** Under the baseline, ASP trailed flat by 3.2 / 3.8 points on the two model slots. Under the context floor, ASP leads by 4.2 / 1.2 points. Among the 345 of 500 rows where expansion actually fired, accuracy rose 2.9 / 1.4 points — a clean causal signal on the dataset the mechanism was designed for.
- **CUAD remains a clear lead but loses some of its ceiling.** ASP still beats flat by 13–16 points EM/F1, but the absolute numbers drop from ~87 EM in the baseline to ~80 EM with the floor active. This is the gating limitation described in §5.3: CUAD's short, span-extraction bundles trigger the floor, and the added parent and sibling clauses dilute the extraction span. The follow-up experiment (the top-1 BM25 margin gate described in §7.1) is designed to leave CUAD untouched.
- **EW1 and EW2 retain a mild positive.** Both are long-form extraction where Exact Match is expected to be 0; ASP's F1 advantage over flat is preserved under the floor.
- **HotpotQA is unchanged by EE4 (no-op).** The baseline HotpotQA gap is a separate problem addressed by other experiments out of scope for this report.
- **Bundle size and latency.** On the single-source benchmarks, ASP context bundles remain dramatically smaller than the flat baseline (CUAD: 240 vs 682 tokens; MAUD with the floor: 825 vs 870; EW1 with the floor: 966 vs 1,283). On the `rr_ctx700` run, ASP latency was 4–63× lower than the flat baseline across single-source datasets. (Caveat: the `rr_ctx700` run executed in a different inference environment from `rr_final`, so latency improvements versus the prior ASP run should not be attributed to EE4-X1 itself — the ASP-versus-flat ratio *within* the same run is the load-bearing comparison.)

6.4 Structure-aware chunking baseline (EE-Struct)

Reviewer feedback on v1 made the point sharply: "ASP cannot rest on beating BM25 + flat hybrid alone. That's the easy baseline. The comparison that actually proves the thesis is ASP against a strong structure-aware-chunking baseline." This is the right question to ask. A sliding-window BM25+vector baseline is a strawman for any system that claims to exploit

document structure — most production RAG stacks already chunk on headings. EE-Struct introduces a third backend, `flat_struct`, that uses ASP's own document-tree parser to produce section-level chunks with heading paths prepended, then runs the *same* hybrid retriever as plain flat. No tree traversal, no parent/sibling expansion, no authority-ordered assembly. Only the chunk boundaries and the heading-path prefix are inherited from ASP. If ASP's win disappears against this stronger baseline, the structural machinery is unjustified.

Method

The structure-aware baseline pipeline is purpose-built for this experiment as a third backend (`flat_struct`). For each document we run the **same document-tree parser ASP uses**; every parsed node becomes one chunk whose text is `"{heading_path}\n{title}\n{content}"`. The chunks are then fed through the *identical* retrieval stack as plain flat: Chroma dense embeddings, BM25 lexical, reciprocal-rank fusion, and a cross-encoder reranker — the BEIR-style hybrid pattern. The structure-aware index is built and stored separately from the sliding-window flat index. Prompts, LLM slots, evaluation harness, item ordering, and n=500 per cell are held constant across all three backends. The only thing that varies is what counts as a chunk: ASP retrieves over tree nodes with traversal and expansion; `flat_struct` retrieves over heading-path-prefixed section chunks with flat hybrid; plain `flat` retrieves over fixed-size sliding-window chunks. By construction this isolates *what* counts as a unit of retrieval from *how* retrieval is done over it.

Results

CUAD (span extraction):

Dataset	Backend	Model	EM%	F1%	avg_ctx_tok	avg_lat_ms	Verdict
CUAD	ASP	Nemotron	73.6	77.5	240	1,820	✅
CUAD	ASP	Mistral	72.2	77.0	240	2,140	✅
CUAD	flat	Nemotron	58.6	62.1	681	2,560	—
CUAD	flat	Mistral	57.6	62.2	681	2,910	—
CUAD	flat_struct	Nemotron	58.4	62.5	1594	3,980	❌
CUAD	flat_struct	Mistral	56.0	60.1	1595	4,310	❌

MAUD (multiple-choice classification):

Dataset	Backend	Model	Acc%	avg_ctx_tok	avg_lat_ms	Verdict
MAUD	ASP	Nemotron	29.8	497	2,210	❌

Dataset	Backend	Model	Acc%	avg_ctx_tok	avg_lat_ms	Verdict
MAUD	ASP	Mistral	27.6	497	2,480	❌
MAUD	flat	Nemotron	31.6	870	2,840	—
MAUD	flat	Mistral	32.0	870	3,120	—
MAUD	flat_struct	Nemotron	21.4	227	1,640	❌
MAUD	flat_struct	Mistral	23.6	244	1,790	❌

Verdict column is relative to plain `flat` as the comparator: ✅ = beats flat by >1pp on the primary metric, ⚠️ = within 1pp, ❌ = trails flat by >1pp.

Reading the results

On CUAD, ASP beats both baselines by roughly +15pp EM (73.6 vs 58.6 vs 58.4 on Nemotron; the Mistral pair tracks within a point). It does so while consuming only ~35% of flat's average context (240 vs 681 tokens) and roughly 70% of flat's latency (1,820 vs 2,560 ms on Nemotron) — simultaneously more accurate *and* about 3× cheaper on retrieved-context cost. `flat_struct` and plain `flat` sit on top of each other to within 1–2pp on both EM and F1, even though `flat_struct`'s average context balloons to 1594 tokens versus flat's 681 — each parsed node becomes one chunk, and parsed nodes are not bounded to the 256-token tiktoken sliding window used by flat. The heading-path prefix gives essentially zero lift on CUAD despite more than doubling the retrieved-context budget. Whatever explains ASP's CUAD advantage, it is not heading-path enrichment.

On MAUD the comparison inverts. `flat_struct` *loses* to plain `flat` by −10.2pp on Nemotron (21.4 vs 31.6) and −8.4pp on Mistral (23.6 vs 32.0). It is worth being explicit that ASP also trails `flat` on MAUD, though by a much smaller margin (−1.8pp on Nemotron, 29.8 vs 31.6; −4.4pp on Mistral, 27.6 vs 32.0). MAUD's M&A agreements parse into many fine-grained clauses, and any retrieval strategy whose chunk boundaries are inherited from the parse tree pays a clause-fragmentation tax — `flat_struct` pays it most acutely because it has no expansion mechanism, ASP pays a smaller version of it because parent/sibling expansion can only widen a bundle so far when the underlying clauses are themselves the unit of meaning. For `flat_struct` specifically, the retriever pulls back tiny isolated clause fragments with no surrounding context (avg 227 tokens, versus flat's 870) — the same context-starvation failure mode that motivated EE4-X1 in §5.1, but worse, because `flat_struct` has no expansion mechanism to widen the bundle once a short chunk is selected.

The net read: heading-path chunking is not table stakes. On the corpus type ASP is built for — clause-heavy contracts where the right answer often lives in a neighbouring clause rather than the one that lexically matched — heading-path chunking actively underperforms plain flat, and even ASP's tree-traversal machinery cannot fully overcome clause fragmentation on MAUD-style multiple-choice classification. The load-bearing parts of ASP are (a) tree traversal, (b) parent/sibling expansion via the floor-expansion

procedure of §5.5, and (c) authority-ordered bundle assembly. The heading-path prefix is a footnote, not the mechanism.

Limitation

`flat_struct` as run here uses only `heading_path + title + content` as chunk text. A stronger structure-aware baseline would add the per-chunk LLM-generated "contextual retrieval" prefixes Anthropic published — paying a one-time index-time LLM cost to attach a short situating summary to each chunk. We expect that variant to close some of the CUAD gap, since the contextual prefix would compensate for the heading-path prefix's failure to disambiguate near-duplicate clauses. It would not, however, fix the MAUD inversion: over-fragmented chunk boundaries are a structural problem, and no amount of per-chunk enrichment can reassemble a clause with its neighbours at query time.

6.5 Model-generalisation check — `llama_base`

A fair reviewer will ask whether the ASP-versus-flat gap is a property of the retrieval method or an artefact of the two specific inference endpoints used for §6.1–§6.4 (the Nemotron-30B and Mistral-119B slots). To separate the two, the `llama_base` run re-executes the same harness — same items, same backends, same evaluation code, **no retrieval change** — with the generation model swapped to a different base model (Llama-3.3-70B, across three parallel slots m2/m4/m6). If the pattern is a retrieval property, it should reproduce; if it was a model artefact, it should collapse.

The within-run ASP-versus-flat delta on the primary metric, per dataset (median across the three slots):

Dataset	Primary metric	ASP	Flat	Δ (ASP − flat)	Direction vs §6.1–§6.4
CUAD	EM%	88.8–90.2	71.0–71.6	**+17.2 to +18.9**	✅ same sign, same +14–18pp band
MAUD	Accuracy %	54.2–57.6	54.6–56.4	−0.4 to **+1.2**	⚠️→✅ near-parity *without* the floor
HotpotQA	F1%	67.1–68.1	72.5–74.4	−4.5 to −7.3	❌ same sign (ASP trails)
EW1	F1%	15.5–21.4	13.4–15.0	+0.5 to **+7.9**	✅ same sign (ASP wins F1)

Dataset	Primary metric	ASP	Flat	Δ (ASP − flat)	Direction vs §6.1–§6.4
EW2	F1%	21.0–22.9	14.4–16.2	**+6.5 to +6.7**	✅ same sign (ASP wins F1)

Every directional finding from the paper slots reproduces on Llama-3.3-70B: ASP wins CUAD by the same +14–18pp EM band, ASP wins the long-form EW1/EW2 datasets on F1, and ASP trails flat on multi-hop HotpotQA by roughly the same 4–7pp F1 margin. The efficiency picture is likewise unchanged — ASP retrieves ~35% of flat's context on CUAD (240 vs 682 tokens) and generates 5–50× faster across every dataset. **The ASP-versus-flat pattern is therefore not model-specific; it is a property of the retrieval mechanism.**

One result moves in ASP's favour: on this base model MAUD reaches near-parity (−0.4 to +1.2pp) *without* the EE4-X1 context floor active, where the paper slots showed ASP trailing flat by 2–4pp under the same no-floor condition (§6.4). This is consistent with the §5.1 diagnosis that MAUD's deficit was context starvation interacting with a weaker generator — a stronger base model tolerates the tighter bundle better — and does not change the case for EE4-X2 (§7.1), which remains the principled fix for the floor firing indiscriminately on CUAD.

The full per-slot scorecard, efficiency table, and no-regression check for this run are recorded in the benchmark report `docs/research/llama_base_report.md`. Model attribution is inferred from the run tag and the multi-slot `.env` example; the result files do not stamp a model name, so this subsection should be treated as a cross-model *replication* of direction and magnitude, not a head-to-head model comparison.

---

7. Open work

The Adaptive Context Floor establishes that structural expansion at bundle-assembly time can repair context starvation on MAUD. Three threads remain open.

1. **CUAD dilution.** The floor currently fires whenever the bundle is short. The follow-up (EE4-X2) replaces that with a **top-1 BM25 margin gate** so the floor only triggers when retrieval is non-discriminating — the MAUD failure mode — and leaves CUAD's already-discriminating retrievals untouched. Margin was chosen over a full variance signal after early prototyping; the rationale is in §7.1.
2. **Floor calibration.** The 345-of-500 fire rate on MAUD is informative but not yet tuned. Whether 700 tokens is the right floor — or whether it should adapt to candidate-clause length — is an open calibration question.

3. **Generalisation.** The mechanism has been validated on one single-source legal MC corpus. Whether it transfers to other near-neighbour classification problems (regulatory categorisation, policy compliance triage) is the natural next benchmark.

7.1 EE4-X2 — Top-1 BM25 margin gate (next experiment)

The v1 reviewer feedback identified a sharp and correct diagnosis: the EE4-X1 floor fires when the assembled bundle is below a fixed token floor (`running_tokens < min_context_tokens`), which is a proxy for retrieval ambiguity, not a measurement of it. On MAUD the proxy happens to align — short bundles really do indicate the ranker is undecided — but on CUAD a short bundle is the signal of a *confident* hit, and the floor misfires there for exactly that reason. EE4-X2 was already the next experiment in the queue when that feedback arrived; the convergence is a useful confirmation that the diagnosis is the right one.

Mechanism

Replace the `running_tokens < min_context_tokens` trigger of §5.5's procedure with a gate computed directly from the BM25 score distribution produced by the §4 scorer. The reviewer feedback on v1 proposed a sibling-score-variance signal; in early prototyping we converged on a simpler **top-1 margin** — how cleanly the best candidate beats the runner-up — because (a) margin is a one-line read of the same score map, (b) it avoids second-moment estimates on small-K samples (K=6), and (c) it discriminates the CUAD-vs-MAUD signal we care about (confident hit vs ambiguous retrieval) with one fewer free parameter than variance. We retain variance as a fallback if margin proves brittle. Formula:

```
top_K          = top retrieval candidates (e.g. K = 6)
s1, s2, …, sK  = BM25 scores, sorted descending
top1_margin    = (s1 - s2) / s1        if s1 > 0
               = 0                     otherwise   (guard for s1 == 0)
trigger expansion iff top1_margin < τ
```

Margin is high when one node dominates (confident hit) and low when the top scores are bunched (ambiguous retrieval), which makes the predicted CUAD/MAUD directions follow directly from the formula rather than being asserted. The guard for `s1 == 0` handles the degenerate case where BM25 returns no overlap at all; the gate defaults to firing, matching today's "expand when uncertain" behaviour.

The threshold τ is auto-calibrated per corpus from a 50-query validation slice — the 25th-percentile margin observed on that slice — so no human tuning is required and the gate logic itself is identical across datasets. A one-off calibration pass per corpus computes τ from this validation slice; this is a build-time cost, not a per-query cost, and it must be re-run when the corpus changes.

Predicted behaviour

- **MAUD (typical case).** Every clause mentions "consideration", "merger", "payment", "closing". The top-6 BM25 scores cluster tightly because the query

terms hit broadly across the document, so `s1` and `s2` are close and `top1_margin` is small → gate fires → the floor-expansion procedure (§5.5) adds parent / sibling context. The bundle ends up roughly where EE4-X1 placed it, so the MAUD result reported in §6.2 should be preserved.

- **CUAD (the misfire case).** The discriminating clause typically owns the query's rare terms ("governing law", "assignment", "non-compete"). Top-1 has a clean BM25 lead over its siblings, `top1_margin` is large → gate does **not** fire → the bundle stays at its retrieved ~240 tokens, no dilution, no expansion cost.
- **No new per-query infrastructure.** The margin signal is read from the ranker the pipeline already runs each query; no second index, no auxiliary LLM call, no extra latency beyond two comparisons over the top-K scores.

Status

Not yet run. The run tag will be `rr_ctx_var`, executed against the same five-dataset, two-model matrix used for EE-Struct, and is expected to land before the funded research engagement begins — so the MAUD-vs-CUAD asymmetry can be reported as *resolved by gate redesign* rather than as a standing open question.

---

8. Appendices

8.1 Glossary of identifiers

This document uses a small number of internal identifiers when referring to experiments and to specific benchmark runs. They are defined below for reader convenience; the complete catalogue of experiments — including unpublished negative results, abandoned variants, and the full programme history — will be released alongside the research paper.

Naming scheme. *Experiment identifiers* follow the form `EE<n>` (a *research epic* — a coherent investigation around one hypothesis) and `EE<n>-X<m>[<suffix>]` (the *m*-th attempt within that epic; a trailing letter denotes a re-run with a single controlled variable changed). A `-X1b` is the second variant of the first attempt under epic `EE<n>`; `-X2` is the second attempt of that epic. The `EE` prefix stands for *experiment epic*; the `X` prefix stands for *attempt*. Experiments referenced narratively without an `-X<m>` suffix denote the epic as a whole.

Run-tag identifiers (e.g. `rr_final`, `rr_ctx700`) label specific benchmark execution batches. The `rr_` prefix marks every run since the round-robin seed-selection patch was merged (`rr_final` was the first stable run with that patch active); the subsequent suffix records the experimental variable for that batch.

Identifiers used in this document.

Identifier	Type	Meaning

Identifier	Type	Meaning
`EE4`	Experiment epic	*Mandatory clause context window* — the research epic investigating context starvation on single-source classification corpora and the use of structural expansion (parent/sibling addition) at bundle-assembly time to repair it.
`EE4-X1`	Experiment attempt	The *Adaptive Context Floor* — the specific experiment described in §5. Adds a `min_context_tokens` floor to bundle assembly with a BFS expand-to-floor procedure. Status: partially promoted (MAUD win, CUAD dilution).
`EE4-X2`	Experiment attempt	The *Top-1 BM25 margin gate* described in §7.1 — replaces EE4-X1's bundle-size trigger with a score-distribution-based trigger, so the floor only fires when retrieval is genuinely ambiguous. Status: not yet executed at the time of writing.
`EE-Struct`	Experiment	*Structure-aware chunking baseline* — the experiment in §6.4 that introduces a third backend, `flat_struct`, to isolate the contribution of heading-path-prefixed chunking from ASP's tree-traversal machinery. Status: complete, reported here.
`rr_final`	Run tag	The clean baseline run reported in §6.1 — round-robin seeding enabled, no context floor. Provides the original ASP-vs-flat

Identifier	Type	Meaning
		comparison without any §5 mechanism active.
`rr_ctx700`	Run tag	The run reported in §6.2 — same harness as `rr_final` but with `min_context_tokens=700` set, i.e. EE4-X1 active.
`rr_struct_baseline`	Run tag	The three-way run reported in §6.4 — ASP versus plain flat versus `flat_struct`, all in the same process on the same items. Used to isolate the marginal contribution of heading-path chunking.
`rr_ctx_var`	Run tag	Reserved label for the EE4-X2 (top-1 margin gate) run referenced in §7.1. Not yet executed at the time of writing.
`llama_base`	Run tag	The model-generalisation run reported in §6.5 — the §6.1 harness re-executed with the generation model swapped to Llama-3.3-70B (slots m2/m4/m6), no retrieval change. Confirms the ASP-vs-flat pattern is not specific to the Nemotron/Mistral slots.

A brief note on what is *not* in this glossary. Experiments touching prompting (chain-of-thought scaffolds, citation formats, source labelling, multi-source synthesis), seed-selection algorithms, recall audits, and retrieval-shape calibration each have their own `EE<n>-X<m>` identifiers and run tags in the internal programme record, but none of them are referenced in this document and so are omitted here. The full programme record, including all negative results, will accompany the research paper.

8.2 Survey matrix

The full set of methods reviewed in §3, with each method's category, whether it uses native document structure as the retrieval signal, and a primary source.

Name	Category	Native structure?	Source
Hybrid BM25 + Dense RAG	Flat hybrid	No	BEIR, Thakur et al. 2021
Anthropic Contextual Retrieval	Flat hybrid + LLM chunk context	Partial (context prefix, no tree)	anthropic.com
Azure AI Search (Text Split)	Cloud platform	No	learn.microsoft.com
Azure AI Search (Document Layout)	Cloud platform	Partial (structure → chunk boundaries)	learn.microsoft.com
AWS Bedrock KB — Hierarchical Chunking	Cloud platform	No (token parent/child)	docs.aws.amazon.com
Google Vertex AI Search — Layout-Aware	Cloud platform	Yes (premium SKU, metered per query)	cloud.google.com
Snowflake Cortex Search	Cloud platform	No	docs.snowflake.com
Databricks Mosaic AI Vector Search	Cloud platform	No	databricks.com
Pinecone	Vector DB	No	pinecone.io
Weaviate	Vector DB	No	weaviate.io
Elastic / ELSER	Search engine	No	elastic.co
ServiceNow AI Search	Enterprise search	Partial (table-rooted)	servicenow.com
Glean Enterprise Search	Enterprise search	Partial (entity graph)	glean.com
Harvey AI Vault	Legal AI	Partial (undisclosed)	harvey.ai
Hebbia Matrix	Agentic doc QA	Partial (page/section iteration)	hebbia.com/matrix
Casetext CoCounsel	Legal AI	No	casetext.com
NetDocuments ndMAX	Legal DMS AI	No	netdocuments.com
RAPTOR	Research	No (synthetic tree by clustering)	arXiv:2401.18059
GraphRAG (Microsoft)	Research / graph	No (entity graph, Leiden community hierarchy)	arXiv:2404.16130
LightRAG	Research / graph	No (LLM OpenIE over	arXiv:2410.05779

Name	Category	Native structure?	Source
		chunks)	
HippoRAG	Research / graph	No (LLM OpenIE + PPR)	arXiv:2405.14831
KAG (Ant Group)	Research / graph	Partial (layout + LLM extraction)	arXiv:2409.13731
Neo4j GraphRAG SimpleKGPipeline	OSS graph	No	neo4j.com
LlamaIndex PropertyGraphIndex	OSS graph	Partial (configurable)	llamaindex docs
LangChain GraphRetriever	OSS graph	Partial (metadata edges)	langchain docs
LlamaIndex DocumentSummaryIndex	OSS hierarchical	No (synthetic summary tree)	llamaindex docs
LangChain ParentDocumentRetriever	OSS hierarchical	No (size split)	langchain docs
Dense X Retrieval (Propositions)	Research	No	arXiv:2312.06648
HiRAG with Rethink	Research	Partial (coarse-to-fine scaffold)	arXiv:2408.11875
MultiHop-RAG	Research / benchmark	No	arXiv:2401.15391
HIRAG (Instruction Tuning)	Research	No (hierarchy in reasoning, not index)	arXiv:2507.05714
ColBERT	Research	No (passage-level)	arXiv:2004.12832
ColPali	Research	No (visual patches)	arXiv:2407.01449
PageIndex	Native tree	**Yes**	github.com/VectifyAI/PageIndex
HiQA	Native tree (cascaded)	**Yes**	arXiv:2402.01767
RDR2	Native tree	**Yes**	arXiv:2510.04293
LAD-RAG	Native layout graph	**Yes**	arXiv:2510.07233
RAG with Hierarchical Knowledge	Native tree	**Yes**	arXiv:2503.10150
MacRAG / Multi-	Research	Partial (granularities	representative arXiv

Name	Category	Native structure?	Source
granularity		from structure)	
LlamaParse	Parser	Partial (metadata only)	llamaindex docs
Azure AI Document Intelligence Layout	Parser	No (feeds flat retriever)	learn.microsoft.com
Unstructured.io (`chunk_by_title`)	Parser	No (flat output)	unstructured docs
Docling HierarchicalChunker	Parser	Partial (metadata only)	docling-project.github.io
Marker	Parser	Partial (tree available, RAG path flat)	github.com/datalab-to/marker
Nougat	Parser	No	github.com/facebookresearch/nougat
pdfplumber	Parser	No	github.com/jsvine/pdfplumber
Adobe PDF Extract API	Parser	No (feeds caller's retriever)	developer.adobe.com
ASP (this work)	Native tree retrieval + structural bundle assembly	**Yes**	—